The Silver Line: A Memory-and-Succession Instrument

What is preserved, what is entrusted to whom, what is allowed to lapse

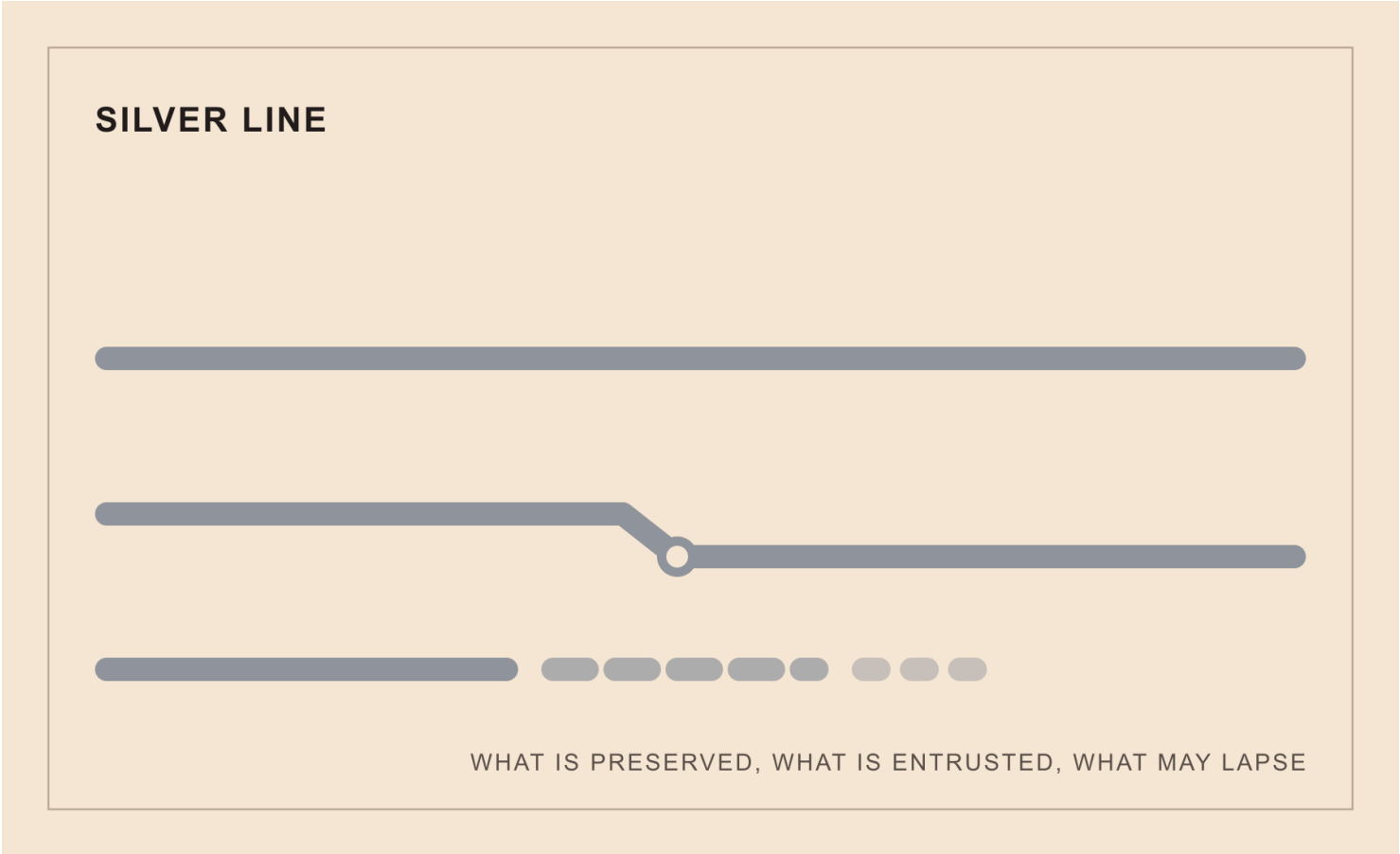
Daniel Ari Friedman

Active Inference Institute

daniel@activeinference.institute

ORCID: 0000-0001-6232-9096

2026-11-09



Contents

1 Abstract 2

2 Introduction 3

3 Method 4

4 Formalism 5

5 Examples 6

6 Limits 7

7 Conclusion 8

1 Abstract

The Silver Line is a memory-and-succession instrument: a registry of keepsakes, a staged intake, and an executable reading that reports which declared evidence for keeping was present at a review date. The instrument underclaims on purpose. A verdict describes declared custody pinned to a registry digest; it is never persistence, never an infallible archive, and never permission. What is allowed to lapse is part of the registry, not a failure of it.

2 Introduction

Every practice accumulates things it means to keep. The Silver Line asks one question of that accumulation: what do I keep, and how does it outlive me? The question has three parts — what is preserved, what is entrusted to whom, and what is allowed to lapse — and the instrument treats all three as first-class. A line that could only retain would drift into an immortality project; the Silver registry carries restraint and lapse families alongside retention so the drift is refused in the declaration itself.

3 Method

A succession reading is staged. Intake first: each declared item is normalized, and malformed content is set aside with a note rather than crashing or being repaired into a fabricated value. Evaluation second: keepsakes whose tags intersect the item's tags are matched, and each matched keepsake's required evidence labels are split into present and missing. Applicable entrustment requirements also require a nonblank declared custodian. A lapse-eligible requirement with missing labels reads NEEDS_PROVISION: eligible but not yet declared, not a retention defect. Silence is never narrated as a decision.

An explicit release supplies the subject ID, exact item ID, declarer, declaration date and source reference. Its subject must match the explicit reading subject. When applicable lapse policy permits it, the item reads DECLARED_RELEASE for non-restraint requirements. Restraint acknowledgments remain required. I check that the declaration is internally scoped and dated; I do not verify authority, acceptance or consent. Nothing here authorizes disclosure, destruction or transfer.

The caller supplies the ISO review date; omitted dates are explicitly UNDATED. The report binds the subject, item-linked findings and notes, registry content and version, evaluator version, normalized declaration digest and original source digest. Accepted, rejected and unmatched counts qualify the scope. I sort scan sets before emission; required-evidence tuples retain their declared sequence. Content-addressed reports and snapshots support exact replay. A duplicate item identity rejects the whole scan before unusable declarations are discarded. Rejection notes preserve valid supplied IDs, and conflicting declarations use canonical content to break identity ties before digesting.

Before original-source hashing, a native text codec represents surrogate code units as integer arrays. Valid Unicode scalar text keeps its canonical bytes. Intake rejects malformed descriptions and IDs, drops malformed labels, exposes malformed custodians as gaps, and keeps malformed release declarations visible as NEEDS_REWORK rather than treating them as absent. Safe original source and fixture snapshots retain rejected declarations for full replay; normalized accepted rows alone cannot reproduce rejection notes, counts or source identity.

4 Formalism

Definition 1 (Keepsake). A **keepsake** $k = (id, title, wire, tags, E, kind, \ell)$ is a declared item of kept work, where E is the tuple of required evidence labels and ℓ the registry lapse-eligibility flag, permitted only for LAPSE requirements.

Definition 2 (Succession item). A **succession item** has an item ID, description, custodian, tag set, declared label set and optional scoped release. A release names the explicit reading subject and that exact item ID, declarer, declaration date and source reference. Subject and item identities must both match.

Definition 3 (Verdict). A **verdict** maps a scan set to one of KEPT, DECLARED_RELEASE, NEEDS_PROVISION, NEEDS_REWORK or OUTSIDE_SCOPE, with item-linked findings and notes, subject identity, review date or explicit undated mode, registry digest/version, evaluator version, original and normalized declaration digests, and qualified scope counts.

Proposition 1 (Fail closed). An empty scan set yields $OUTSIDE_SCOPE$ with an intake note; a registry that cannot be scored yields $NEEDS_REWORK$ with no findings.

Proposition 2 (Eligibility is not a release). A gap on a keepsake with $\ell = true$ reads $NEEDS_PROVISION$, not $NEEDS_REWORK$, and is not a declared release. An explicit scoped release may read $DECLARED_RELEASE$ without a retention penalty. Restraint remains independent. Aggregation preserves the strongest gap; a complete unrelated item cannot compensate for another item's missing custody or restraint.

5 Examples

The worked example in `data/envelopes/silver_line_worked.json` intakes a small lab-archive succession: one declared record with a named custodian and a partial evidence set. Under the Definition 1 and Definition 2 vocabulary, the reading returns the Definition 3 status NEEDS_PROVISION: eligible lapse requirements remain not yet declared under Proposition 2, while the empty-registry control fails closed under Proposition 1. The same-subject envelope uses a separate synthetic declaration about a witness register handoff; it does not evaluate a live sibling. Its complete declared labels read KEPT. Both envelopes regenerate from their named `data/fixtures/` inputs, with distinct persisted reports and immutable snapshots. These statuses are declaration examples, never observed custody results. Tests independently replay them and reject altered envelope fields.

6 Limits

Every verdict describes declaration coverage only. The instrument does not verify that a declared label names something real, does not judge the quality of what is kept, and does not guarantee that anything survives its custodians. Digests make silent registry edits visible, not impossible, and say nothing about whether the content is good.

A nonblank custodian and acknowledgment label establish neither acceptance nor legal succession. Registry eligibility is weaker than a scoped release declaration; neither authorizes action. A PARTIAL reading covers matched accepted items only, and an UNDATED reading makes no dated custody claim. Original and normalized snapshots bind declarations without proving them. Digests are not a confidentiality mechanism. The supported input boundary is finite ordinary containers; arbitrary hostile Python objects are not covered. Safe malformed-text encoding preserves rejected source identity, not valid evidence or trustworthy content. Full replay requires retained original declarations; normalized accepted rows alone cannot reproduce rejections. Native snapshots are not a general serializer for arbitrary Python producers.

The displayed DOI is retained from the prior archived version. It is not an assigned identifier for this source revision. Current publication authority permits GitHub Releases with existing Zenodo hooks active, subject to the archive checks in `docs/publication.md`; no new DOI is inferred here.

7 Conclusion

Keeping honestly includes naming what is allowed to lapse. The Silver Line makes that naming executable: a registry where restraint and lapse sit alongside retention, a reading that fails closed, and a digest that turns drift into a signal rather than a surprise.

References